Exception Handling

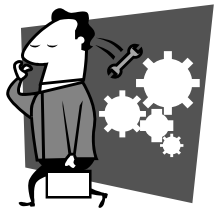
Contents of lecture 8:

1. **Exception handling in Java**
   - Overview of program error handling
   - Exceptions within Java

Different types of errors

During the first lecture a number of common complication errors were identified, and an approach towards removing logical/run-time errors explored.

The importance of removing such errors cannot be overstated, especially for commercial programs where the image of the company may be adversely impacted by a poor product (which also has an effect upon the perceived quality of new products from that company).



Whilst it may be possible to reduce the number of logical/run-time errors to an acceptably small number, it is important to realise that a number of other problems (many of which may be external to your program) may still arise.

Examples of such problems include:

- The program tries to write some data to a disk file, but an abnormal I/O termination occurs as the disk has no free space.
- The program tries to connect to a particular Internet address, but a connection cannot be established.
- Your program instantiates some more objects, however, when the Java VM tries to allocate memory for the new objects, it finds the computer has run out of free memory.

The root causes of all the above problems are external to the program that reported the error. Nonetheless, a robust and well-written program will try to deal with the problem, e.g. by informing the user of the error, or exiting gracefully.

To summarize, errors may be of the following types:

1. Complication errors
2. Logical/run-time errors arising from incorrect code within the program
3. Logical/run-time errors originating from a source external to the program.

Methods of handling the last two types of error are explored within this lecture.

Errors within Java

Logical or run-time errors within Java are signified, or represented, through the use of exceptions. An exception is nothing more than an object that contains information on the type of error that occurred, in the same way that a GUI event is nothing more than an object containing information on the type of event that occurred.

Hence, whenever a disk file cannot be successfully accessed, Java will generate an Exception object containing details of the problem. Likewise, whenever a network problem occurs, Java will package up details of the problem into an exception.

Apart from using exceptions to represent errors external to the program, Java also employs exceptions for aberrant code within the program. For example, whenever an invalid array index is accessed, or an illegal class-cast attempted, Java will generate an exception.

In what follows, the traditional (which is to say, non-exception handling approach) to error capture and recovery is firstly outlined. Next, the notion of exception handling is introduced, and finally Java specific coverage provided.

Traditional error detection/correction

The goal

Assume we have a robust application, entailing that the program is well written, scrutinises the validity of all user input, checks the success of disk I/O, monitors the state of network connections, etc. Such a program is likely to be well received by its users.

Once such a program detects a problematic situation, e.g. invalid user input, etc., then to be robust the program must satisfactorily handle the situation, recovering from the problem if possible, informing the user, or in a worst-case scenario gracefully quitting.

Traditional approach to goal attainment

The traditional approach, which is to say the approach routinely employed before exception handling was introduced, consisted of error correction and detection code interspersed throughout the program

In particular, errors were dealt with at the place in which they occured (e.g. each time some memory was allocated a check was made to ensure that it was successfully obtained, a check was made before each mathematical operation which might result in a division by zero, etc.).

In many situations this type of error detection/recovery is appropriate, and permits the programmer to easily see what type of error detection/correction is being performed. However, this approach is also problematic, in that the code becomes 'polluted' with error recovery code (making it more difficult for the programmer to understand the purpose of the code). Additionally, the programmer might forget to include error detection/correction code at certain points within the program.

A C++ example follows (error detection / correction code shown in bold):

```
int LoadWordPhonemeListFile( char *szDictName )
{
        // Try to open the specfied input file.
        ifstream fileInput( szDictName, ios::in );
        if( !fileInput ) return( 0 );

        while( !fileInput.eof() )
        {
                ptroDictionaryEntry = new sDictionaryListEntry;

                // Check if the entry could be allocated.
                if( ptroDictionaryEntry == NULL )
                {
                        cout << "\nOut of memory."; exit( -1 );
                }
…
```

Exception Handling

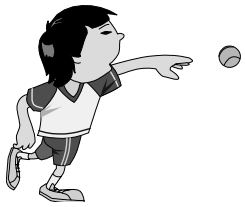
Exception handling supports the notion of generalized error processing, whereby, error correction code can be separated from the main body of code and placed in a number of exception handlers. In other words, exceptions are for changing the flow of control when something important or unexpected, normally an error, occurs. In particular, control is diverted to another part of the program that can try to cope with the error, or at least 'gracefully die'. Some terminology used within exception handling follows:

Description	Java
An unexpected error that occurs during runtime.	Exception
The process by which an exception is generated and passed to the program.	Throwing
Capturing an exception that has just occurred and executing statements that try to resolve the problem	Catching
The block of code that attempts to deal with the exception (i.e. problem).	Catch clause, or catch block
The sequence of method calls that brought control to the point where the exception occurred.	Stack trace

The following piece of code illustrates a simple exception handler.

```
try
{
        file = new DataInputStream(
                new InputStream( fileName ) );
}
catch( IOException e )
{
        System.out.println( "\nDisk problems" );
}
```

An exception handler consists of two core sections, called blocks. The **try** block encloses some code which might throw an exception (i.e. generate an error). The **catch** block contains the error handling code, i.e. determines what should be done if an error is detected.



Hence, exception handling provides a means of separating error handling from any code that might result in errors. In many situations this is beneficial as it produces 'cleaner' code. Consider the following example:

Opening a file using C++ and Java

C++

```
ifstream inputFile( inputFileName );
if( !inputFile )
        cout << "\nError";
```

Java

```
try {
        inputFile = new DataInputStream( name );
}
catch( IOException e )
{
        System.out.println( "\nError" );
}
```

If anything, the C++ code is more simple than the Java exception handling code. However, consider the situation where we wish to open three files:

C++

```
ifstream inputFile( inputFileName );
if( !inputFile )
        cout << "\nError";

ifstream dataFile( dataFileName );
if( !dataFile )
        cout << "\nError";

ifstream data2File( data2FileName );
if( !data2File )
        cout << "\nError";
```

Java

```
try {
        inputFile = new DataInputStream( inputFile );
        dataFile = new DataInputStream( dataFile );
        data2File = new DataInputStream( data2File );
}
catch( IOException e ) {
        System.out.println( "\nError" );
}
```

In this particular case the exception handling code is more compact and understandable, not suffering from the clutter of error checks. As the program size increases, and/or the amount of error checking increases, this benefit becomes more significant.

It is important to note, that for small programs exception handling does not reduce the amount of work involved when handling errors. Instead, the main benefits of exception handling are only observed when a sizeable program must be constructed.

When to use exception handling

Exception handling is suited to a number of different types of error handling, as follows:

- To process exceptional situations – i.e. those errors that cannot be reasonably predicted, or those situations expected to occur infrequently.
- To process exceptions from libraries, classes, etc. which cannot be expected to handle their own exceptions (e.g. network connections, etc.)
- On large projects, to handle error processing in a uniform, project wide, manner. This would be accomplished through the use of global exception handlers.

Common sense is normally adequate to determine if exception handling (global or local) or traditional error checking should be employed.

> **Local exception handlers**: Defined as those catch blocks that immediately follow the try block in which the exception was thrown.
> **Global exception handlers**: Catch blocks found higher up the call hierarchy, intended to offer generalized error handling.
>
> Both terms are explored later within this lecture.

When not to use exception handling

In certain cases, exception handling is not appropriate. Most importantly, exception handling should only be employed for exceptional events – i.e., errors that occur infrequently. For example, after asking the user to enter a value between 0 and 10, it would be inappropriate to use exception handling to deal with any out-of-range errors, instead more traditional error checking should be used.

Likewise, exception handlers should not be used to process key presses, mouse clicks, network messages, etc. (which are better confined to event/interrupt processing).

Exceptions: The complete story

Consider the following code fragment:

```
try
{
        Code();
}
catch( exception1 )
{
        ErrorProcessing1();
}
...
catch( exceptionN )
{
        ErrorProcessingN();
}
…
finally
{
        Finally();
}
```

The code that may generate an exception is enclosed within a **try** block (the **try** statement meaning 'try these statements and see if you get an exception). The **try** block is immediately followed by one or more **catch** blocks. Each **catch** block specifies the type of exception it can catch and contains an exception handler for that exception. In effect, the **catch** block says 'I will handle any exception that matches my argument'.

Lastly we have an optional **finally** block that is executed regardless of whether or not an exception occurred. The **finally** block is always executed, even if one of the other blocks (try or catch) contained a return statement. As such, the **finally** block is ideal for those situations where some code must always be executed (e.g. any housekeeping tasks).

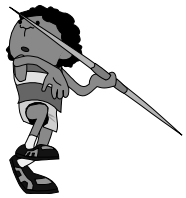
Generating an exception

Exceptions are generated by one of two different mechanisms:

1. The Java VM automatically generates an exception, e.g. it runs out of memory, encounters a division by zero, etc.
2. The programmer explicitly causes an exception to be thrown by using the **throw** keyword. An example follows:

```
public double quotient( int num, int den) throws DivideByZeroException
{
        if( den == 0 ) throw new DivideByZeroException();
        return (double)num/den;
}
```

In the vast majority of programs, the programmer need only worry about exceptions thrown automatically by Java. It is only when programs need to introduce exceptions specific to the needs of that program that new exceptions must be created.



The point at which a **throw** is executed is termed the throw point.

Once an exception occurs, the block in which the exception is thrown *expires* and control cannot resume from the throw point. This is termed the *termination model of exception handling* (which Java implements). In the *resumption model of exception handling* control returns to the point at which the exception was thrown and execution resumes.

If necessary, additional information can be added to the thrown exception, and passed onto the exception handler (such as the exact location at which the error occurred).

Consider the following example, which divides two user entered numbers (and employs exception handling to deal with any problems).

```
public class DivideByZeroException
                    extends ArithmeticException
{
        public DivideByZeroException()
        { super("Attempted to divide by 0"); }
}
```

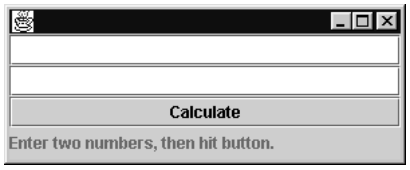
ArithmeticException is a built in exception.

Store some textual info on the exception

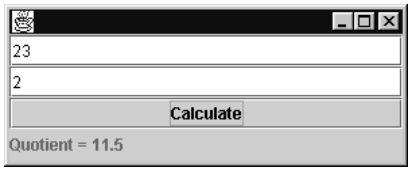
```
import java.awt.*; import java.awt.event.*; import javax.swing.*; import javax.swing.event.*;
import java.util.*;

public class ExceptionExample extends JFrame
{
        JTextField number1; JTextField number2;
        JButton calc; JLabel result;

        // Construct the GUI
        public ExceptionExample()
        {
                number1 = new JTextField( 10 );
                number2 = new JTextField( 10 );
                result = new JLabel( "Enter 2 numbers:" );
                calc = new JButton( "Calculate" );
                calc.addActionListener( new ActionListener()
                { public void actionPerformed( ActionEvent event )
                        { determineResult(); }
                } );

                Container content = getContentPane();
                content.setLayout( new GridLayout( 4, 1 ) );
                content.add( number1 ); content.add( number2 );
                content.add( calc ); content.add( result );

                setSize( 300, 120 ); setVisible( true );
        }

        // Start the program running
        public static void main( String args[] )
        {
                ExceptionExample example = new ExceptionExample();
                example.addWindowListener( new WindowAdapter()
                        { public void windowClosing( WindowEvent e )
                                        { System.exit( 0 ); }
                        } );
        }
```

```
        // Determine the result of the division
        private void determineResult()
        {
                try
                {
                        // Extract numbers
                        int iNum1 = Integer.parseInt( number1.getText() );
                        int iNum2 = Integer.parseInt( number2.getText() );

                        // Determine the quotient
                        double dResult = quotient( iNum1, iNum2 );
                        result.setText( "Quotient = " + dResult );
                }
                catch( NumberFormatException e )
                {
                        result.setText( "Please enter integer values." );
                }
                catch( DivideByZeroException e )
                {
                        result.setText( e.toString() );
                }
        }

        // Perform the division
        public double quotient( int iNum, int iDen )
                throws DivideByZeroException
        {
                if( iDen == 0 ) throw new DivideByZeroException();

                return (double) iNum/iDen;
        }

}
```

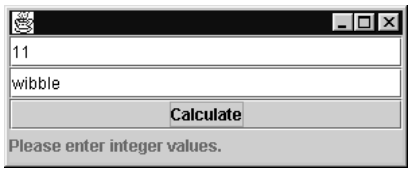
The above example prompts the user to enter two numbers. Once the 'Calculate' button is hit the second number is divided into the first, and the quotient reported on the screen.

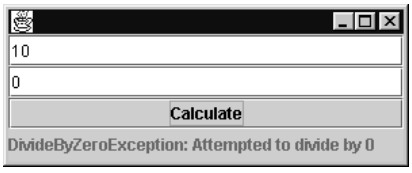


The example not only defines a new type of exception, extending the built in ArithmeticException class, but also introduces code that explicitly throws an exception (i.e a user generated exception is thrown, as opposed to the built-in exceptions which are thrown from within the Java libraries, e.g. NumberFormatException.



Note, if the **try** block successfully executes, and no exceptions are thrown, then all the exception handlers are skipped and execution resumes with the first statement after the exception handler. If a **finally** block follows the last **catch** block, then the code contained within the **finally** block is executed.



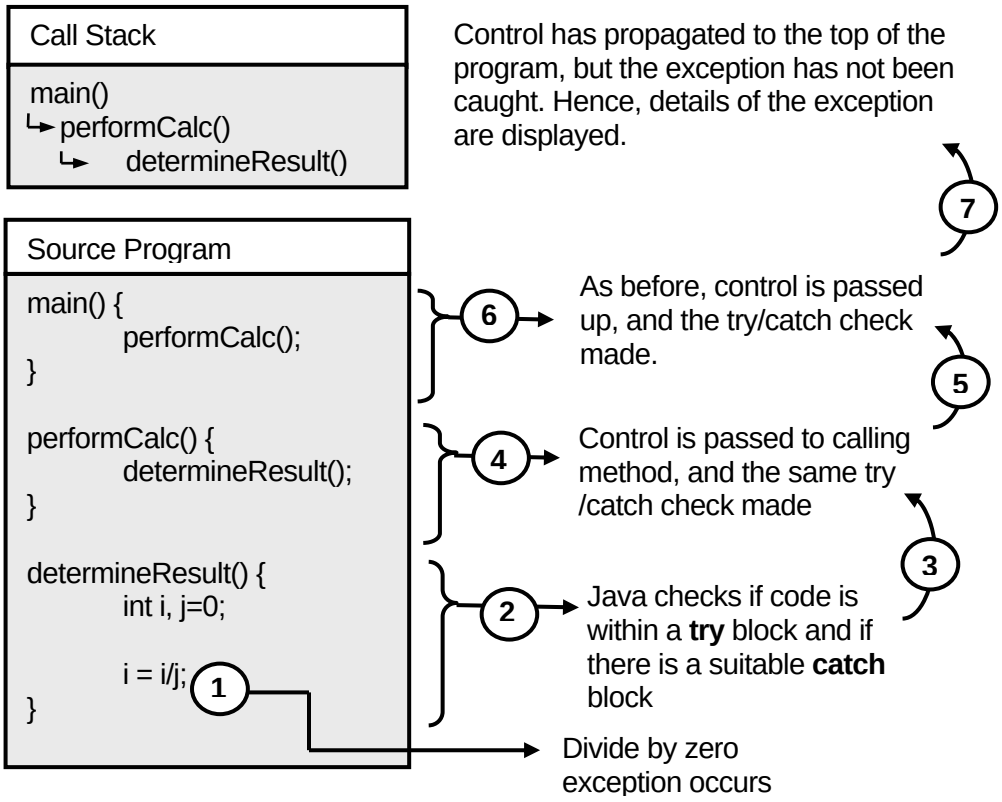


Catching an exception

Whenever an exception is generated, Java immediately transfers control to the **catch** block(s) attached to the **try** block within which the exception occurred.

If no handler matches the type of exception that was thrown within the **catch** block, then Java attempts to find a match for the exception in a 'try-catch' block further up the call chain, enclosing the method call that brought control to the point where the exception occurred. The following example shows how this process operates:

The main() method calls the performCalc() method, which in turn calls the determineResult() method, inside which an exception is generated.



At point 1, an exception was thrown, with the code that threw the exception not contained within a **try** block. What happens next is the same as if the code was contained within a **try** block, but no matching **catch** block could be found. In particular, the flow of control abruptly leaves the method, and a premature return is performed (back up the call stack to performCalc).

If the code within the calling method (performCalc) is enclosed within a **try** block it is then checked to see if it contains a matching exception handler. The process is repeated, either until an exception handler is found, or until the top-most method in the call chain (e.g. main) is reached.

If no handler can be found for the exception then non-GUI based applications quit, whilst GUI based applications and applets return to their normal event processing (depending upon the severity of the exception).

Note that it is possible several exception handlers will exist for a given exception (e.g. a global, catch-all, exception handler might have been defined, as well as more local exception handling). In this case, the first exception handler that matches the exception type is executed.

Structured in this way, global error handling can be straightforwardly introduced, as illustrated below:

```
try
{
        doALotOfProcessing();
}
catch( SomeCommonError )
{
        doErrorProcessing();
}
catch( AnotherCommonError )
{
        doSomeOtherErrorProcessing();
}
```

In this case, the try block encloses a call to a method doALotOfProcessing which might contain a sizeable portion of code. The code might generate a number of exceptions, which are dealt with by the global exception handlers – thereby removing the problem of overlooked or forgotten error checking within the code.

Miscellaneous

Catching an exception, then what?

What should the exception handler do when an exception has been caught? Possibilities include rethrowing the exception, converting the exception into a different type of exception and rethrowing that, recover from the exception and resume processing (possibly transferring control back to the method that generated the exception), etc.

An exception might be rethrown if the exception handler determines that it cannot handle the exception. It is normally rethrown in the same manner it was originally thrown, i.e. through the use of the **throw** keyword.

printStackTrace/getMessage

The Throwable class, which all exceptions inherit from, defines two methods that can be used to return information about the exception that occurred.

The getMessage() method returns a textual message about the exception, whilst the printStackTrace() method returns the call stack leading up to the exception (i.e. what methods were called, etc.).

Throwable classes within Java

Any object of the Throwable class can be thrown (this also applies to any subclass of Throwable). Java defines two direct subclasses of the Throwable class: Exception and Error. Exceptions are intended to cover any problems that the program can be reasonably expected to deal with, whilst errors cover serious system problems that should not be caught by the program.

Exceptions within Java

The following exceptions are defined within Java:

Exception:
ClassNotFoundException
IllegalAccessException
InstantiationException
InterruptedException
NoSuchMethodException

RunTimeException
- ArithmeticException
- ArrayStoreException
- ClassCastException
- IllegalArgumentException
  - IllegalThreadStateException
  - NumberFormatException
- IllegalMonitorStateException
- EmptyStackException
- NoSuchElementException
- IndexOutOfBoundsException
  - ArrayIndexOutOfBoundsException
  - StringIndexOutOfBoundsException
- NegativeArraySizeException
- NullPointerException

IOException
- EOFException
- FileNotFoundException
- InterruptedException
- UTFDataFormatException
- MalformedURLException
- ProtocolException
- SocketException
- UnknownHostException
- UnknownServiceException

AWTException

Errors within Java

The following table lists the different types of Error that might be thrown within Java (Reminder: Error's are serious problems that the programmer should not try to catch and resolve).

Error:
- LinkageError
  - ClassCircularityError
  - ClassFormatError
  - IncompatibleClassChangeError
    - AbstractMethodError
    - IllegalAccessError
    - InstantiationError
    - NoSuchFieldError
  - NoClassDesFoundError
  - UnsatisfiedLinkError
  - VerifyError
- ThreadDeath
- VirtualMachineError
  - InternalError
  - OutOfMemoryError
  - StackOverflowError
  - UnknownError

Practical 8

After this lecture you should explore the eighth practical pack which should enable you to investigate the material in this lecture.

Learning Outcomes

Once you have explored and reflected upon the material presented within this lecture and the practical pack, you should:

- Understand that traditional error handling and exception handling provide two different approaches towards the detection and correction of errors. Furthermore, have knowledge of the strengths and weaknesses of each approach and be capable of determining which approach is appropriate given a straightforward problem description.
- Understand the terminology used throughout exception handling.
- Have knowledge of the common built-in exceptions provided by Java. Additionally, understand how and why user-defined exceptions may be created.
- Understand the process through which exceptions are generated by either the Java VM or the programmer. Additionally, understand the procedure by which an exception is passed to an appropriate handler. Finally, understand the types of error processing than can be attempted once an exception has been caught.
- Understand the distinction between local and global exception handlers, and the conditions under which each event handler type should be used.
- Be capable of creating a local exception handler to deal with common exceptions occurring within Java programs.

More comprehensive details can be found in the CSC735 Learning Outcomes document.